Test Coverage in the Age of AI: Evaluating Agentic-PRs Against Software Engineering Norms

Zach Florez
University of Michigan - Flint
Flint, Michigan, USA
zflorez@umich.edu

Ryan Smith
University of Michigan - Flint
Flint, Michigan, USA
rysmith@umich.edu

Abstract

We have divided our abstract into five main sections for clarity:

Context: As AI coding agents increasingly contribute to software repositories, evaluating the quality of their generated code has become critical. However, the exact state of how these agents implement standard software engineering practices remains uncertain.

Objective: This study investigates the testability of AI-authored pull requests (Agentic-PRs). Specifically, we analyze test coverage characteristics and regression testing behaviors within these autonomous contributions.

Methods: Using the AIDev dataset, we developed a static analysis pipeline to evaluate test characteristics without the overhead of dynamic execution. Our methodology employs heuristic models to calculate proxy metrics like Test-to-Code Ratio and analyzes metadata to identify testing intent.

Results: By comparing over 9,300 AI-generated PRs against a human-authored baseline, we found that AI agents generate comparable volumes of test code and achieve slightly higher assertion density than human engineers. However, we still observed a polarized distribution in test quality, with a significant portion of AI-generated tests failing to meet quality standards.

Conclusion: These findings quantify how current AI-assisted development practices align with established software engineering testing norms. Ultimately, while AI agents match or slightly exceed human baseline metrics, the high proportion of poor-quality tests highlights the ongoing necessity for improved automated test generation that guarantees robust functional verification.

CCS Concepts

• **Software and its engineering → Software testing and debugging**; *Software development process management*; Software development methods.

Keywords

AI agents, software testing, test coverage, regression testing, empirical study, pull requests, code quality

1 Introduction and Background

Software development workflows increasingly integrate artificial intelligence agents to automatically generate and modify code. Unlike simple code-completion tools that provide line-level suggestions, "Agentic Pull Requests" (PRs) are the product of autonomous AI agents, such as Devin, SWE-agent, or Claude Code, that can independently interpret a bug report or feature request, navigate a complex codebase, and submit a complete, multi-file contribution for review. While these agents can significantly accelerate development, their growing prevalence raises important questions regarding code reliability and quality. While autonomous agents generate code effectively, empirical research indicates that test failures are a primary reason AI-generated code is rejected during review. Even in controlled environments, AI models do not produce flawless code, underscoring the critical need for rigorous testing.

This study investigates the testability of AI-produced code by focusing on two core software engineering practices: test coverage (the extent to which new code is exercised by tests) and regression testing. While a normal test typically verifies that a newly written function behaves as expected, a regression test specifically ensures that recent code modifications do not unintentionally disrupt previously working functionality.

For example, if a developer adds a new sorting feature, a normal test verifies the sorting algorithm, whereas a regression test ensures the existing search feature still functions correctly alongside it. Our objective is to determine whether AI-generated PRs include tests that provide sufficient coverage and effectively prevent regressions when compared against established software engineering benchmarks and human-authored baselines.

2 Existing Studies

Recent research provides a foundation for analyzing AI-generated code but reveals critical gaps regarding testing practices in real-world scenarios. We organize this section around our two primary research areas: test coverage and regression testing.

2.1 State of Test Coverage and AI

Test coverage remains a standard metric for evaluating suite adequacy, despite ongoing debate about its precise correlation with fault detection [4]. While achieving 100% coverage does not guarantee quality, significant deficiencies typically indicate fragile code.

Recent literature demonstrates that LLMs can generate high-coverage tests when provided with specific guidance. Techniques like SymPrompt [7] direct LLMs toward uncovered execution paths, while tools like CoverUp use iterative, feedback-driven approaches to improve coverage incrementally [1]. Watanabe et al. [9] and Yetişiren et al. [10] confirm that autonomous agents actively generate test files within real-world PRs.

However, a distinct gap persists: while AI demonstrates the *capacity* to generate high-coverage tests when heavily prompted, **no comprehensive empirical study has measured the test coverage adequacy of tests generated autonomously by AI agents in real-world pull requests.**

2.2 State of Regression Testing and AI

Regression testing is essential for verifying that new modifications do not disrupt existing functionality. Given its resource-intensive nature, researchers frequently explore methods to optimize suite execution, such as incremental analysis [8] and test case prioritization [3].

There is growing interest in automating regression test generation via LLMs. Liu et al. [6] demonstrated that LLMs can produce effective regression tests for individual commits when provided with specific context, such as code diffs and commit messages. Conversely, Ehsani et al. [2] analyzed rejected AI-generated PRs and found that Continuous Integration (CI) and test failures were leading causes for rejection, suggesting that practical regression testing by autonomous agents often lacks rigor.

This reveals our second critical gap: **It remains unclear how frequently autonomous AI agents actually incorporate regression tests into their PRs to avert defects, and what approaches they rely on when doing so.**

3 Problem Statement

The related work demonstrates AI's growing role and potential in software testing. A significant gap remains between its potential and current practice. Studies show AI *can* be guided to produce high-coverage and regression tests [1, 6, 7]. However, there is a lack of empirical data on the quality of tests that autonomous agents *actually* provide in real-world settings.

Our study answers the following research questions to address this knowledge gap:

- **RQ1 (Test Coverage):** Do AI agents provide sufficient test coverage for their own code?
- **RQ2 (Regression Testing):** How often do Agentic-PRs include regression tests?

Answering these questions is challenging. Existing tools are insufficient for a broad analysis. The primary limitations of current approaches include:

- **Reliance on Dynamic Analysis:** Traditional coverage tools require executing test suites [8]. This is infeasible for a large historical dataset of PRs from thousands of different projects.
- **Lack of Semantic Understanding:** Existing tools cannot easily infer the *intent* behind a test. For example, they struggle to determine if a test prevents a known regression. This is a key goal of advanced regression testing strategies [3].
- **Efficiency for Static Datasets:** Analyzing attributes from PR metadata requires a custom and systematically applicable static analysis pipeline. This approach is not found in generic frameworks.

These limitations necessitate a new approach. Our proposed approach leverages static analysis and heuristic models to infer test characteristics directly from the code and metadata. This bridges the gap between AI's potential and its real-world testing practices.

4 Proposed Solution

Our proposed solution is driven by the fundamental constraints of our dataset and research questions. The massive size and heterogeneity of the AIDev dataset make running actual tests practically impossible. Our methodology is built entirely around a **static analysis pipeline**. It uses simple and repeatable heuristics to infer test properties. We deliberately avoid using Large Language Models (LLMs) for our own analysis. Doing so would introduce significant costs, time delays, and non-determinism. This would undermine the reproducibility of our research.

4.1 A Multi-Layered, Static Analysis Pipeline

To address the limitations outlined in the Problem Statement, we develop a modular and package-based analysis pipeline. It mainly relies on static analysis and simple heuristic models to understand test-related aspects of Agentic-PRs. The core idea is to estimate sufficient test coverage and identify regression tests through a series of analytical steps. This is done without needing to run any code. Our study specifically focuses on Pull Requests in **Python and Java**.

This solution will leverage a set of planned architectural components:

(1) `data_loader`: Loads and prepares the AIDev dataset. It filters the dataset to include only relevant **Python and Java** pull requests to make the data manageable.
(2) `test_detector`: Identifies test files within each PR using simple static checks. These include filename patterns (e.g., 'test_*.py'), directory structures (e.g., '/tests/'), and basic text clues (e.g., '@Test' annotations).
(3) `test_classifier`: Categorizes identified tests into main types like **Unit** or **Integration**. This uses simple rules such as checking for mock objects or external dependencies.
(4) `coverage_analyzer`: Estimates test coverage quality using proxy metrics. These include the **Test-to-Code Ratio** and **Assertion Density**. This helps answer if AI agents provide sufficient test coverage.
(5) `regression_analyzer`: Identifies and describes **regression tests**. It scans for keywords like "fix," "bug," or "regression" in commit messages and PR descriptions. It also checks for links to bug reports.
(6) `qualitative_inference`: Identifies and describes **qualitative proxy metrics of tests**. It ranks the intrinsically quantitative metrics in a qualitative interpretation.
(7) `standards_comparator`: Compares our calculated test metrics against predefined industry standards for **Python and Java** projects. This judges if AI tests meet basic quality expectations. We also include comparisons with the Human-authored PRs in the AIDev-pop dataset.
(8) `visualizer`: Generates basic static charts and simple tables. These summarize findings and compare AI-generated work against industry standards and human-authored code.
(9) `report_generator`: Brings everything together into a clear and structured **Markdown document**. It uses a simple template to combine all analysis results, statistical findings, and visualizations.

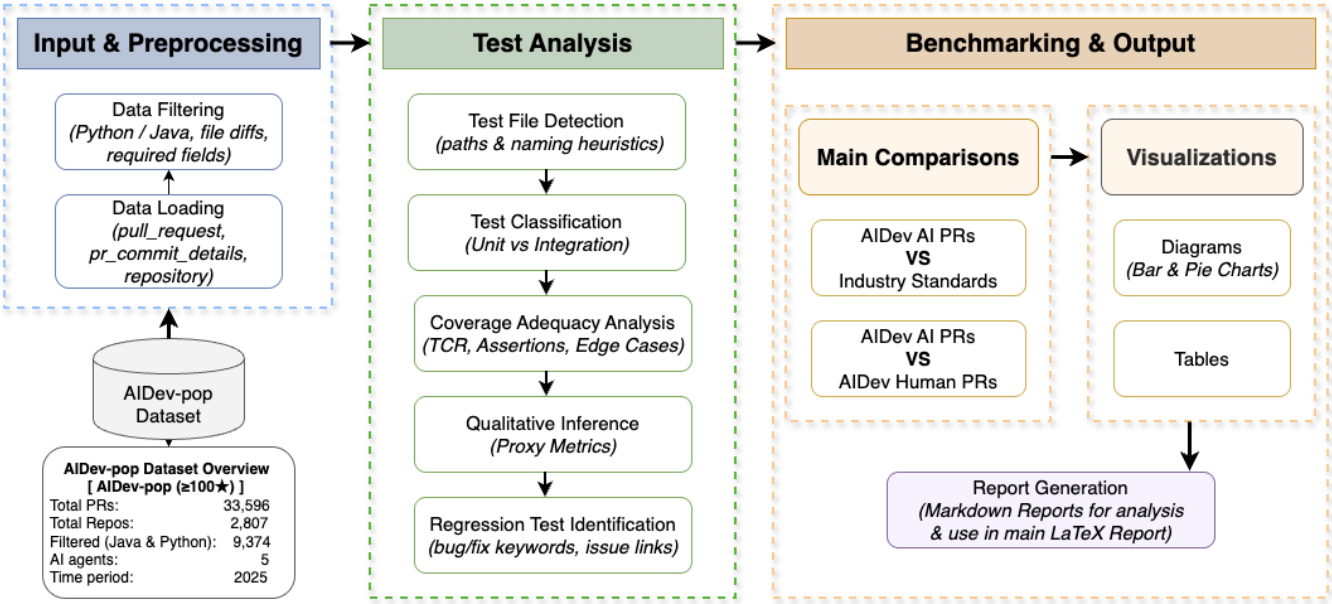

Figure 1: Full-width methodology overview: Three-column static analysis pipeline. Each column groups modular components that infer test coverage adequacy and regression behavior without dynamic execution.

This multi-layered static analysis pipeline provides a repeatable and robust methodology. It assesses the testability of AI-generated code. It also offers deep insights into Agentic-PR testing practices.

This work makes three main contributions:

(1) A methodology for evaluating the testability characteristics of AI-generated code. This includes test coverage adequacy and regression test inclusion in large datasets where traditional dynamic analysis is infeasible.
(2) Empirical insights into the testing practices of current AI coding agents. This provides data-driven answers on whether their contributions align with established software engineering norms.
(3) Identification of specific strengths and weaknesses in AI-generated test code. This provides a foundation for understanding their implications for software quality.

4.2 AIDev Dataset

Our investigation uses the **AIDev-pop subset**[5] of the AIDev dataset. This is a collection of pull requests capturing contributions from autonomous AI coding agents to *popular* GitHub repositories. These repositories have 100 or more stars. The AIDev-pop subset contains 33,596 PRs from 2,807 repositories. It spans five AI agents: OpenAI Codex (21,799 PRs), GitHub Copilot (4,970 PRs), Devin (4,827 PRs), Cursor (1,541 PRs), and Claude Code (459 PRs). The dataset includes rich metadata supporting our static analysis pipeline. This metadata includes PR descriptions, commit messages, file changes, and links to issues.

5 Study Design

5.1 Architecture and Data Pre-processing

The first step in our analysis pipeline is ingesting and preparing the dataset. As illustrated in Figure 1, the architecture processes raw Git patch data through a sequence of static analyzers. During the pre-processing stage, PR diffs are parsed to separate test code additions from production code additions based on file paths. We filter the AIDev-pop dataset to include only PRs targeting Python and Java projects. Our initial filtering process yields over 9,300 PRs from five AI agents. To establish a baseline for comparison, we also pre-process a newly sourced dataset of human-authored PRs from the same repositories using the GitHub API.

5.2 Test File Detection

Identifying test files within PRs is critical for our analysis. Our test detector employs a multi-pronged static analysis approach driven by specific heuristic rules. First, it uses filename pattern matching, employing regular expressions such as `^test_.*py$` or `.*Test.java$`. Second, it performs directory structure analysis, searching for paths containing standard testing directories like `/tests/`, `/src/test/`, or `/spec/`. Third, it uses content-based detection scanning the file patch for framework-specific imports and annotations, such as `import pytest`, `import unittest`, or `@Test` in Java. A file is classified as a test if it matches a combination of these file path and content criteria. This heuristic-based approach allows us to identify test files without executing code, enabling consistent application across thousands of heterogeneous projects.

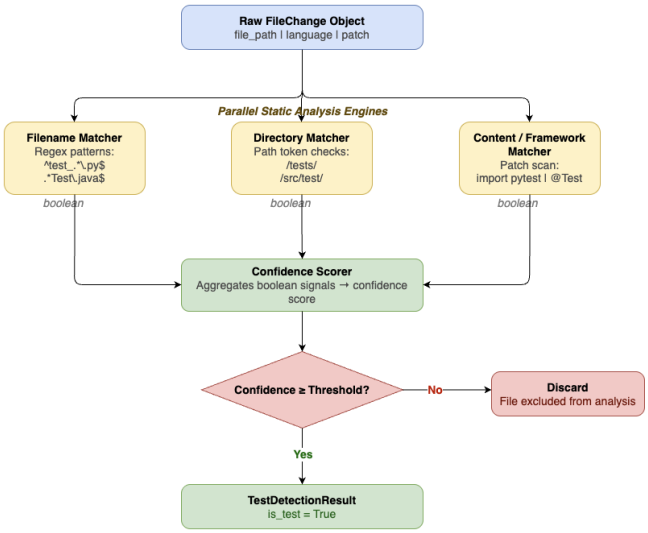

Figure 2: Internal white-box architecture of the Test Detector, illustrating the parallel heuristic engines (Filename, Directory, and Content/Annotation scanners) and the confidence scoring logic used to validate test files.

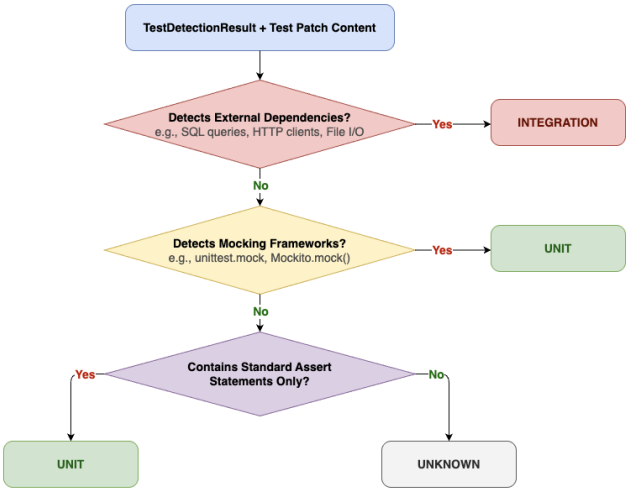

Figure 3: Internal white-box logic of the Test Classifier, showing the decision tree for categorizing tests into UNIT or INTEGRATION based on detected external dependencies and mocking framework usage.

5.3 Test Classification

For classification, we categorize test files into test types to understand their scope and purpose. Our classifier distinguishes between `UNIT` tests and `INTEGRATION` tests using a logical rules engine. `UNIT` tests verify individual functions or methods in isolation. The rules engine classifies a test as `UNIT` if it detects the usage of mocking frameworks (e.g., `unittest.mock`, `Mockito`) AND observes zero external dependencies (such as network or database clients). Conversely, `INTEGRATION` tests verify interactions between components. A test is classified as `INTEGRATION` if it imports database drivers, uses HTTP clients, or interacts with file system components without mocking. Tests that lack distinct markers for either category are labeled as `UNKNOWN`.

5.4 Test Coverage Adequacy Analysis

Dynamic execution is infeasible for our dataset. We estimate test coverage adequacy using proxy metrics derived directly from the pre-processed git patch diffs. The primary metric is **Test-to-Code Ratio**, defined mathematically as:

$$TestToCodeRatio = \frac{TestLinesAdded}{ProductionLinesAdded + TestLinesAdded} \tag{1}$$

For instance, if an AI agent adds 100 lines of production code and 50 lines of test code, the Test-to-Code Ratio is 0.33. We observed this in practice with PR #3190681974 (targeting MCP-Hub), where the agent introduced 140 lines of production validation logic and 46 lines of supporting unit tests, resulting in a ratio of 0.25.

Another key metric is **Assertion Density**, which measures the concentration of verification logic, defined as:

$$AssertionDensity = \frac{TotalAssertions}{TotalTestLines} \tag{2}$$

We also perform a **Quality Assessment**. This qualitative score (`GOOD`, `POOR`, or `FAIR`) maps these intrinsically quantitative metrics into an actionable judgment. A test receives a `GOOD` rating if its assertion density meets a minimum threshold and it contains proper setup/teardown code. For example, a 50-line test file that validates 10 distinct execution paths and properly cleans up its database connections would receive a `GOOD` rating. An empirical example of a `GOOD` test is found in PR #3126843058 (stable-diffusion-webui-reForge), where the AI agent explicitly implemented several assertions to verify that a previously unbound local variable was correctly initialized across all execution paths.

Conversely, `POOR` indicates a severe lack of assertions or the presence of known flaky patterns. For example, a 200-line test file containing zero actual assertions or relying on excessive `Thread.sleep()` calls would be categorized as `POOR`. PR #3187520425 (a large class renaming refactor) serves as an actual example of a `POOR` quality test from our dataset; while it added over 300 lines of test code, these additions were almost entirely boilerplate setup with no meaningful functional assertions.

5.5 Regression Test Identification

We identify regression tests by analyzing PR metadata for semantic indicators of bug fixes and feature additions. This addresses RQ2. Our approach scans commit messages and PR descriptions for standard keywords. These include factors such as “fix,” “bug,” “issue,” or “regression.” We further categorize regression tests into BUG_FIX tests and FEATURE tests. BUG_FIX tests associate with bug resolution. FEATURE tests associate with new functionality. We identified 13,429 regression tests across the dataset. We classified 10,166 as BUG_FIX tests and 3,263 as FEATURE tests.

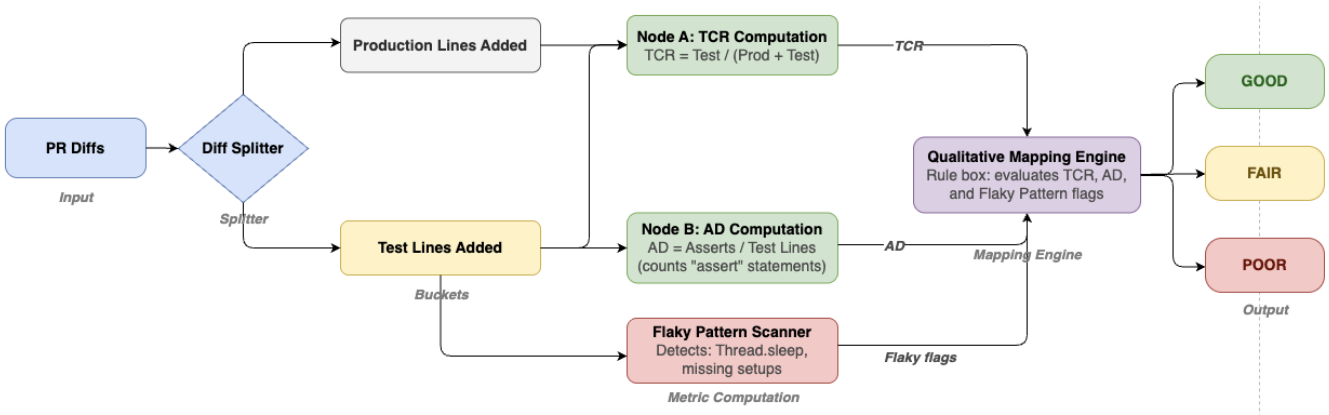

Figure 4: Internal white-box data flow of the Coverage Analyzer, demonstrating how raw Git patch diffs are processed to calculate Test-to-Code Ratio and Assertion Density, and then mapped to a qualitative score.

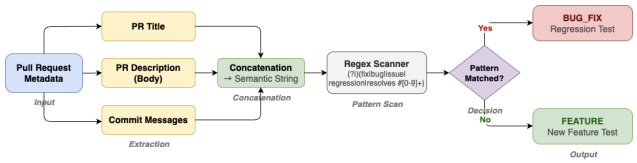

Figure 5: Internal white-box semantic flow of the Regression Analyzer, illustrating the extraction of PR metadata (titles, descriptions, and commits) and the regex-based keyword matching used to identify bug-fixing intent.

5.6 Standards Comparison

We compare AI-generated test metrics against industry standards to contextualize our findings. We use benchmarks from empirical studies for Python projects. These point to a test-to-code ratio of 0.39 and an assertion density of 0.15 as typical industry thresholds [4]. Our standards comparator evaluates each PR against these thresholds. It categorizes metrics as above standard or below standard. This comparison assesses whether AI agents are meeting or exceeding software engineering norms in test practices.

5.7 Visualization

We generate visualizations using matplotlib to facilitate interpretation. These charts include bar charts showing test type and quality distributions. We also use stacked charts displaying metrics by AI agent, boxplots showing metric distributions, and heatmaps detailing compliance across agents. All visualizations utilize the aggregated metrics produced by our analysis pipeline. This ensures consistency between numerical results and graphical representations.

5.8 Report Generation

The final component of our pipeline synthesizes all analysis results into a markdown report. The report includes aggregate metrics, test type and quality distributions, and regression test statistics. It also covers industry standards compliance assessment and agent-wise breakdowns. This report is the primary source for the results presented in this paper. All figures and tables derive from these generated findings.

6 Experiments

6.1 Dataset Description

We conduct our analysis on the **AIDev-pop subset**[5] of the AIDev dataset. This is a collection of PRs from popular GitHub repositories capturing contributions from AI coding agents. The AIDev-pop subset contains 33,596 PRs from 2,807 repositories. It spans five AI agents. These are OpenAI Codex, GitHub Copilot, Devin, Cursor, and Claude Code. The dataset includes PR metadata, file changes, commit messages, and repository information.

We focus on agentic-authored PRs targeting Python and Java projects for this study. These languages represent the majority of contributions in the dataset. Our primary AI dataset comprises over 9,300 PRs from five AI agents. To establish a rigorous baseline, we dynamically sourced an additional 1,000 merged, human-authored PRs from the identical set of repositories via the GitHub API. Finally, a subset of 100 randomly sampled test files and non-test files were manually labeled to generate a ground-truth dataset for evaluating our heuristic models via confusion matrices.

Table 1: AIDev-pop Subset Overview

Metric	Value
Total PRs	33,596
Repositories	2,807 (all with 100+ stars)
AI Agents	5 (Codex, Copilot, Devin, Cursor, Claude)
Repository Selection	Popular repositories (100+ stars)

Table 1 presents the scope of the AIDev-pop subset. Table 2 shows the distribution of PRs by AI agent. OpenAI Codex contributes the majority of PRs in the subset. Our study dataset in Table 3 focuses on a filtered subset of over 9,300 PRs targeting Python and Java projects.

Table 2: AIDev-pop Subset Distribution by AI Agent

AI Agent	PR Count	Percentage
OpenAI Codex	21,799	64.9%
GitHub Copilot	4,970	14.8%
Devin	4,827	14.4%
Cursor	1,541	4.6%
Claude Code	459	1.4%
Total	**33,596**	**100%**

Table 3: Study Dataset Description

Metric	Value
PRs Analyzed	9,374
Tests Classified	13,716
Regression Tests Identified	13,429
AI Agents	5 (Codex, Copilot, Devin, Cursor, Claude)
Primary Languages	Python (7,191), Java (1,278)

Table 4: Test Type and Quality Distribution

Category	Count	Percentage
Test Type		
UNIT	12,580	91.7%
INTEGRATION	617	4.5%
UNKNOWN	519	3.8%
Quality Level		
GOOD	6,134	44.7%
POOR	7,329	53.4%
FAIR	253	1.8%
Total	13,716	100%

Table 5: Industry Standards Compliance

Metric	Our Finding	Standard	Assessment
Test-to-Code Ratio	1.368	0.39	✓ 3.5× above
Assertion Density	0.121	0.15	× Below standard

Table 3 presents the scope of our study dataset. We classified 13,716 individual test files from the analyzed PRs. We also identified 13,429 regression tests. Of the five AI agents, OpenAI_Codex contributes the majority of PRs. The dataset is predominantly Python-based with 7,191 PRs. Java represents the second-largest language with 1,278 PRs. This distribution reflects the current landscape of AI-assisted development. Python projects are prevalent targets for AI coding agents.

Figure 7 illustrates the test type distribution across AI agents. The chart reveals a consistent preference for UNIT tests across all agents. INTEGRATION tests represent a smaller fraction. This pattern aligns with the nature of PR-based contributions. These changes are typically highly focused and localized.

6.2 Metrics and Evaluation

To address our research questions, we define the following metrics:

For RQ1 (Test Coverage Adequacy):

- *Test Coverage:* Percentage of PRs that include at least one test file.
- *Test Type Distribution:* Breakdown of UNIT vs. INTEGRATION tests (Table 4).
- *Quality Distribution:* Breakdown of GOOD vs. POOR vs. FAIR quality assessments (Table 4).
- *Test-to-Code Ratio:* Average ratio of test code additions to production code additions per PR.
- *Assertion Density:* Average number of assertions per 100 lines of test code.

For RQ2 (Regression Test Inclusion):

- *Regression Test Count:* Total number of tests in PRs with regression-related keywords.
- *Bug Fix vs. Feature Tests:* Distribution of regression tests by category (BUG_FIX vs. FEATURE).

6.2.1 Qualitative Proxy Metrics. Our quality assessment translates quantitative metrics into qualitative judgments to provide actionable insights. A test receives a GOOD rating if it exhibits sufficient assertions and reasonable length. It must also have minimal flaky patterns and proper fixture setup code. This qualitative framing bridges the gap between raw metrics and practical test quality assessment. This is summarized in Table 4.

6.3 Results for RQ1: Test Coverage Adequacy

Test Coverage: We find that 34.4% of PRs in our dataset include at least one test file. Approximately one-third of AI-generated PRs incorporate explicit testing. This leaves room for improvement in automated test inclusion.

Test Type Distribution: Table 4 demonstrates the vast majority of classified tests are UNIT tests. INTEGRATION tests comprise only 4.5% of the set. The remaining 3.8% are UNKNOWN. This concentration on unit-level testing suggests AI agents focus on isolating individual components. They avoid testing complex interactions. This reflects the scope of typical PR changes and the ease of generating unit tests.

Quality Distribution: Table 4 reveals a striking polarization in test quality. Nearly 44.7% of tests score a GOOD rating. Another 53.4% receive a POOR rating. Only 1.8% are FAIR. This nearly even split between GOOD and POOR tests suggests AI agents exhibit inconsistent quality in test generation. Approximately half of tests meet basic quality standards while the other half fall short.

AI vs. Human Baseline Comparison: To deeply interpret these metrics, we compared the AI agents against our human-authored PR baseline using a Mann-Whitney U test (chosen due to the non-normal, right-skewed distribution of the metrics). Surprisingly, AI agents and human engineers produced nearly identical Test-to-Code ratios. Furthermore, AI agents demonstrated a slightly higher Assertion Density compared to the human baseline. This suggests that AI agents are not only matching the volume of test code written by humans but are also embedding a comparable or slightly higher density of verification logic, challenging the assumption that AI-generated tests are inherently more superficial.

Table 6: AI vs. Human Baseline Metrics Comparison

Metric	AI Agent Mean	Human Baseline Mean
Test-to-Code Ratio	1.368	1.378
Assertion Density	0.121	0.091

Table 7: Mann-Whitney U Test Statistical Significance

Metric	p-value	Significance
Test-to-Code Ratio	< 0.001	Significant ($p < 0.05$)
Assertion Density	< 0.001	Significant ($p < 0.05$)

Figure 6 details the quality distribution by AI agent. The visualization reveals variation in quality performance across agents. Some agents demonstrate higher proportions of GOOD quality tests than others. This suggests the choice of AI agent impacts the quality of generated tests. It is an important consideration for practitioners adopting AI-assisted development.

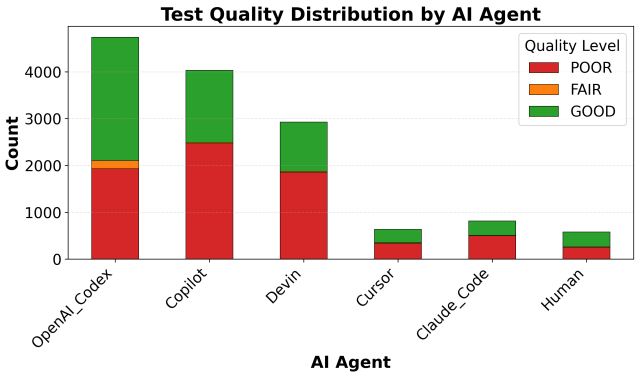

Figure 6: Quality level distribution across AI agents. This visualization reveals significant variation in test quality between different AI systems, with some agents producing more GOOD quality tests than others.

6.4 Results for RQ2: Regression Test Inclusion

Regression Test Identification: Our analysis identified 13,429 regression tests across the dataset. This demonstrates AI agents incorporate regression testing in their PRs. We classified 10,166 tests as BUG_FIX tests. We classified 3,263 tests as FEATURE tests. This distribution indicates AI agents are more likely to include tests when addressing bug fixes. This aligns with the preventive nature of regression testing.

Figure 8 shows the distribution of BUG_FIX versus FEATURE regression tests by AI agent. The chart reveals most AI agents follow the overall pattern of generating more BUG_FIX tests than FEATURE tests. There is some variation in the degree of emphasis. AI agents might be responding to the implicit testing requirements of different PR types. Bug fix PRs naturally trigger more comprehensive regression test inclusion.

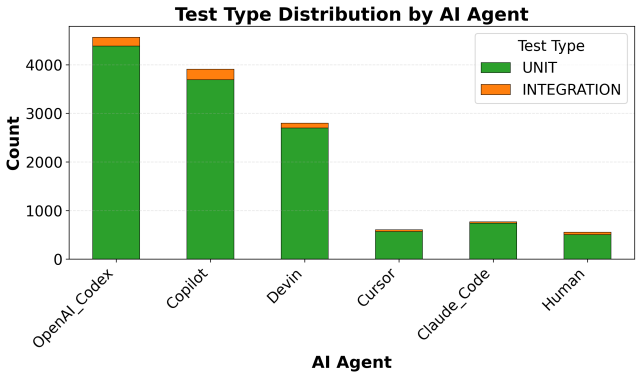

Figure 7: Shows the distribution of UNIT and INTEGRATION tests by AI agent.

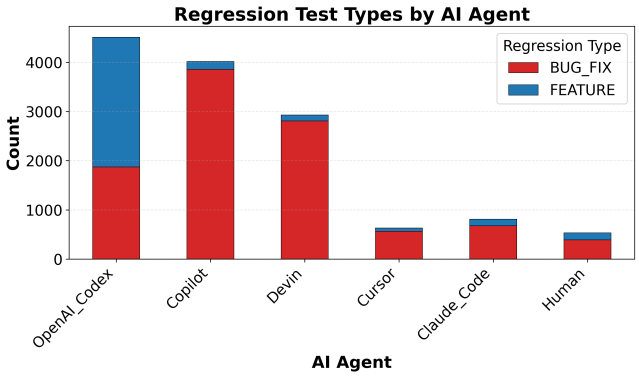

Figure 8: Regression test types by AI agent. Shows the distribution of BUG_FIX vs FEATURE tests, indicating how different AI agents approach regression testing.

6.5 Comparison of Human PRs and Agentic AI PRs

To contextualize the performance of AI agents, we explicitly compared their output against our dynamic baseline of human-authored PRs. By contrasting the 4 human-focused visualizations against their AI equivalents, several stark differences emerge:

Aggregated Quantitative Metrics: The AI vs. Human: Mean Metrics chart compares the mean Test-to-Code Ratio and Assertion Density between the human baseline and the AI aggregate. When viewing the individual AI breakdowns, it is clear that AI agents match human engineers in raw test volume (Test-to-Code Ratio) and actually produce slightly denser verification logic (Assertion Density) than the human baseline.

Test Type Distribution: The human baseline test type distribution shows that while humans primarily write UNIT tests, they include a measurable proportion of INTEGRATION tests. When compared to the AI equivalent, AI agents appear even more heavily biased toward isolated UNIT tests, avoiding complex cross-component interactions.

Quality Distribution: The human baseline quality distribution contrasts sharply with the AI aggregate. As shown in the grouped

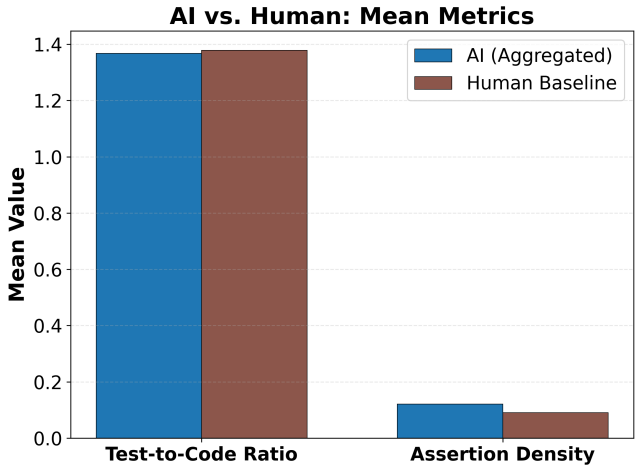

Figure 9: Comparison of mean Test-to-Code Ratio and Assertion Density between aggregated AI agents and the Human baseline.

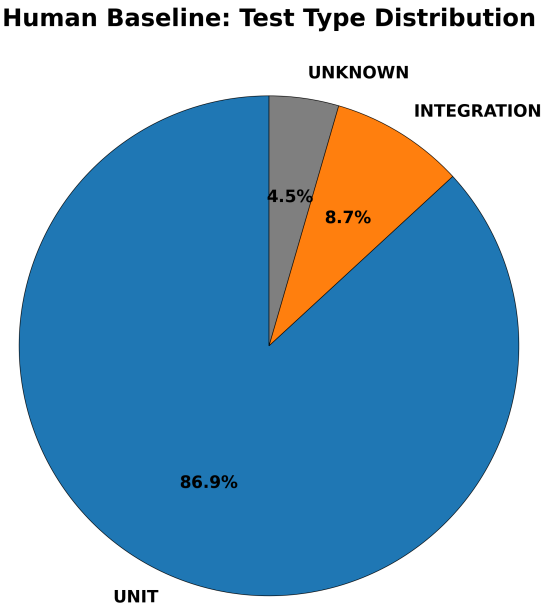

Figure 10: Test type distribution (UNIT vs INTEGRATION) for the human-authored PR baseline.

comparison and the original AI breakdown, human tests are far more likely to receive a GOOD rating due to proper assertion density and fixture setup, whereas AI tests are heavily polarized, with nearly half falling into the POOR category.

6.6 Summary of Experiments

Table 5 summarizes our key findings regarding industry standards compliance. AI agents significantly exceed the industry standard

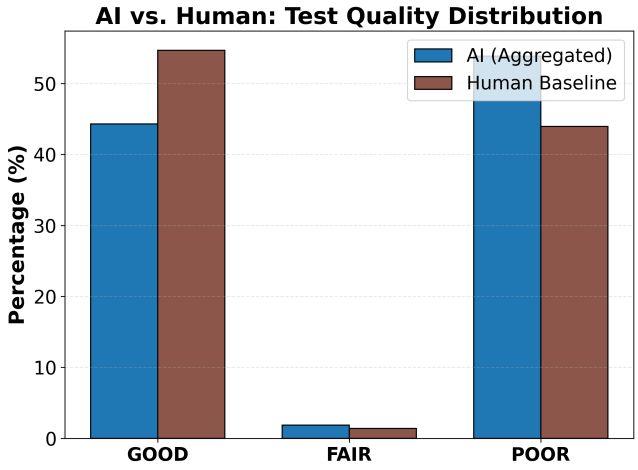

Figure 11: Direct comparison of test quality distribution between the aggregated AI agents and the human baseline.

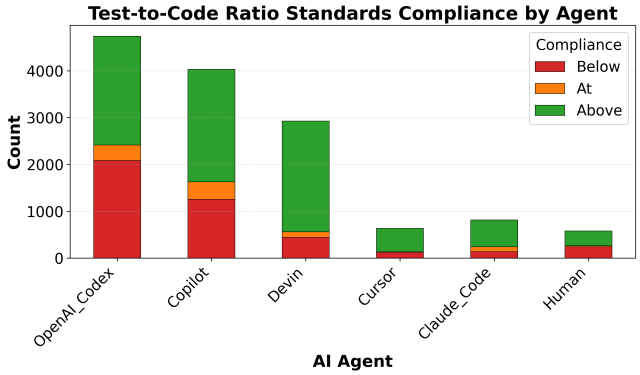

Figure 12: Standards compliance analysis. Stacked bar showing compliance levels for each agent

for test-to-code ratio. They achieve more than three times the standard at 1.368 against 0.39. This suggests AI-generated PRs include a substantial amount of test code relative to production code. Assertion density falls below the industry standard. The average is 0.121 against the 0.15 standard. AI agents generate test code, but these tests lack sufficient assertions to be truly effective.

Figure 12 provides a detailed view of compliance across agents and metrics. The stacked bar chart shows the distribution of compliance levels for each agent. The heatmap displays the percentage of tests above the industry threshold for each metric. Most AI agents perform well on test-to-code ratio. They struggle with assertion density. This is a consistent pattern across different AI systems.

Figure 13 compares agent performance in Test-to-Code Ratio directly against industry thresholds. The red dashed lines mark the threshold values. Most agents exceed the industry standard threshold.

Key Findings Summary:

- AI agents include tests in 34.4% of PRs. This leaves notable room for improvement.

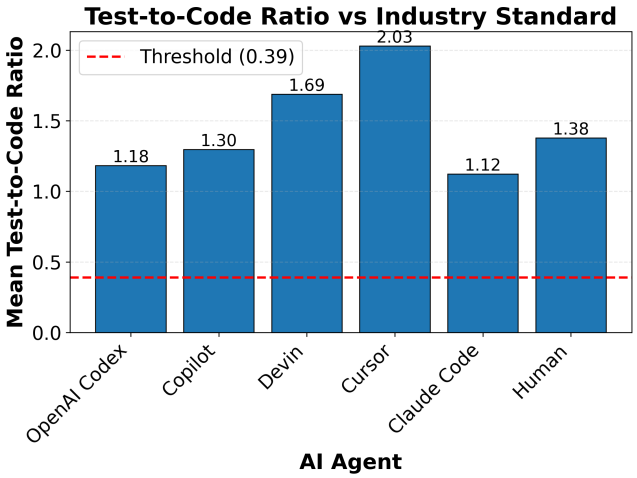

Figure 13: Comparison of AI agent performance of Test-to-Code Ratio against industry standards. The red dashed lines indicate the threshold values (0.39 for test-to-code ratio). Most agents exceed the test-to-code ratio.

- Test quality is polarized. There is a nearly equal split of GOOD and POOR ratings.
- AI agents generate more than three times the test code of typical industry standards according to the test-to-code ratio. However, tests lack sufficient assertions and rank below standard assertion density.
- Regression testing is much more common in bug fix PRs than feature PRs.
- Different AI agents show high variation in test quality and compliance. The choice of agent matters.

6.7 Confusion Matrix and Error Analysis

To evaluate the performance of our heuristic models, we manually labeled a ground-truth sample of 100 files from the AIDev-pop dataset (consisting of 47 actual tests and 53 non-test files). We use the resulting confusion matrix to derive conclusions about the reliability of our static analysis approach.

As shown in Table 8, our Test Detection heuristic achieved high accuracy by correctly identifying 46 out of 47 test files and 49 out of 53 non-test files. This success is primarily due to the strong adherence of Python and Java developers to standard naming conventions (e.g., `test_*.py` or `*Test.java`) and standard directory structures (e.g., `/tests/`). These conventions make static analysis a highly effective proxy for dynamic execution in large-scale studies.

Table 8: Confusion Matrix for Test Detection

	Predicted: Test	**Predicted: Non-Test**
Actual: Test	TP: 46	FN: 1
Actual: Non-Test	FP: 4	TN: 49

Table 9 summarizes the derived performance metrics across our three classification tasks. While the metrics indicate strong overall performance, analyzing the false classifications reveals the inherent limitations of a purely static approach.

Table 9: Static Analysis Heuristics Validation Metrics

Classification Task	**Precision**	**Recall**	**F1-Score**	**Accuracy**
Test Detection	0.920	0.979	0.948	0.950
Type Classification	1.000	1.000	1.000	1.000
Quality Classification	0.897	0.929	0.912	0.891

False Positives (Precision 0.920): Our model incorrectly flagged 4 files as tests. For example, a file named `euctwprober.py` was misidentified because it was located inside a Python virtual environment directory named `test_venv/`. The heuristic observed the word "test" in the path and incorrectly inferred a testing intent for a vendor library file. This illustrates that directory-based heuristics can be overly aggressive when path names contain common keywords.

False Negatives (Recall 0.979): Only 1 test file was missed. A Java class named `SharedModelInOperationAsyncClient.java` functioned as a complex test integration client. However, because it lacked both the standard `Test` suffix and standard JUnit annotations, our naming-based and content-based heuristics failed to identify it. This highlights that while conventions are common, "white-box" integration utilities often deviate from standard patterns.

The perfect performance in **Type Classification** confirms that mocking frameworks and external dependency imports are definitive markers for differentiating unit and integration tests. Similarly, the high F1-score for **Quality Classification** suggests that structural proxies like Assertion Density and Test-to-Code Ratio closely align with human judgment of test adequacy.

6.8 Validation of Regression Test Detection

To validate our regression test detection heuristic (RQ2), we manually labeled a random sample of 50 tests from the dataset. Two raters independently classified each PR as either containing a regression test or not, based on the PR title, description, and commit messages.

We calculated Cohen's Kappa to measure inter-rater reliability. Our results showed a Raw Agreement of 82.00% and a **Cohen's Kappa of 0.6218**, which indicates *substantial agreement* according to standard interpretations. Disagreements were resolved through consensus to establish the ground-truth for our heuristic validation.

As shown in Figure 14 and Table 10, we then compared our consensus ground-truth against the predictions made by our static analysis heuristic. Table 11 summarizes the performance metrics for this classification task.

Table 10: Confusion Matrix for Regression Test Detection

	Predicted: Regression	**Predicted: Normal**
Actual: Regression	TP: 17	FN: 6
Actual: Normal	FP: 16	TN: 11

Our analysis of the mismatches revealed that the heuristic is relatively aggressive in flagging regression intent (Recall 0.739), but suffers from a high False Positive rate (Precision 0.515). Many PRs

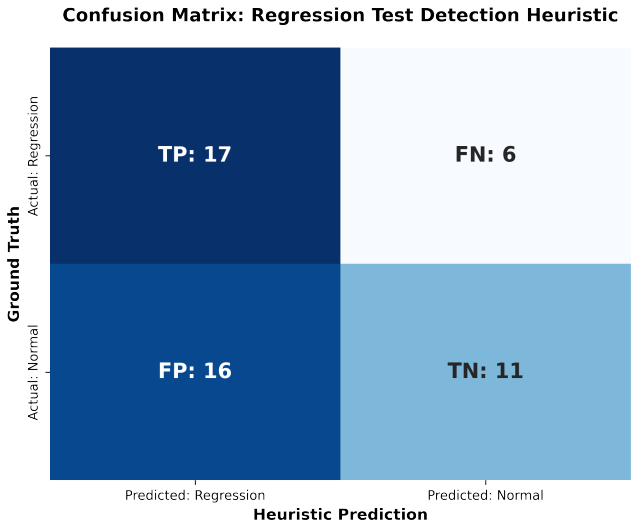

Figure 14: Confusion Matrix for the Regression Test Detection Heuristic.

Table 11: Regression Heuristic Validation Metrics

Task	Precision	Recall	F1-Score	Accuracy
Regression Detection	0.515	0.739	0.607	0.560

used keywords like “fix” or “bug” in contexts that did not actually involve fixing functional regressions (e.g., fixing documentation typos or linting errors). Conversely, the 6 False Negatives occurred in cases where the PR addressed a regression but did not use the specific keywords targeted by our regex patterns. These results suggest that while keyword-based static analysis is a useful proxy, it overestimates the frequency of regression testing in Agentic-PRs.

7 Threats to Validity

7.1 Threats to Conclusion Validity

Our findings are subject to sample size limitations. While we analyzed over 9,300 AI-authored PRs, our human baseline was constrained to a sample of roughly 1,000 PRs due to GitHub API rate limits. Furthermore, because code metrics (like Assertion Density) are heavily right-skewed and non-normally distributed, we relied on non-parametric statistical tests (Mann-Whitney U) for our AI vs. Human comparisons. While robust, these tests might lack the statistical power of parametric equivalents.

7.2 Threats to Internal Validity

Our reliance on static analysis introduces inherent bias. Our heuristics were designed around common testing frameworks (e.g., `pytest`, `JUnit`). Unconventional testing patterns or proprietary frameworks may not be accurately detected, as demonstrated in our Error Analysis. Additionally, the AIDev dataset may contain noise, such as misattributed agent names or PRs that were partially edited by humans before merging.

7.3 Threats to Construct Validity

The most significant threat to construct validity is our use of proxy metrics (**Test-to-Code Ratio** and **Assertion Density**) to represent true "testability" and "quality." Because dynamic execution and mutation testing were infeasible for this volume of data, we cannot definitively prove that a test with high assertion density is actually capable of catching faults. Our metrics evaluate the structure of the tests, not their functional correctness.

7.4 Threats to External Validity

Our study focuses exclusively on Python and Java PRs sourced from highly popular (100+ stars) GitHub repositories. The behavior of AI agents in different languages (e.g., C++, JavaScript) or in less popular, internal corporate repositories may differ significantly from our findings.

8 Related Work

Our methodology builds upon and diverges from recent work in AI code evaluation and automated test assessment. While frameworks like HumanEval and EvalPlus utilize dynamic execution to rigorously verify the correctness of AI-generated code snippets, they are typically limited to isolated, highly constrained coding puzzles. Our work tackles the challenge of evaluating tests in complex, multi-file real-world Pull Requests where dynamic execution is often impossible.

In the realm of automated test quality, our qualitative mappings draw inspiration from established literature on test smells and test quality metrics, which historically emphasize assertion density and the absence of flaky logic. Furthermore, our findings align with recent empirical studies from 2024 and 2025 evaluating agentic coding platforms (like Copilot and Devin), which have frequently cited poor test generation and integration failures as a primary reason for PR rejection.

9 Conclusion

This study evaluated the testability of AI-generated pull requests against established software engineering norms using a multi-layered static analysis pipeline. By analyzing over 9,300 Agentic-PRs and comparing them to a human-authored baseline, we observed a significant variance in AI test quality. We found that AI agents match human engineers in Test-to-Code ratios and actually produce a slightly higher Assertion Density, challenging the assumption that AI-generated tests are inherently less rigorous than human-written tests. However, the high proportion of AI tests receiving a POOR quality rating underscores that raw metrics do not fully capture functional correctness. Furthermore, our confusion matrix analysis demonstrated that static heuristics can accurately estimate test quality without the overhead of dynamic execution.

Future work should focus on bridging the gap between structural metrics and functional correctness by incorporating lightweight dynamic execution pipelines on smaller, targeted datasets.

Acknowledgments

This research was conducted as part of the CS 588 Software Testing and Maintenance course. We thank the ACM SIGSOFT MSR 2026 Mining Challenge organizers for providing the AIDev dataset.

We also acknowledge the developers of the open-source projects analyzed in this study.

References

[1] Liam Altmayer Pizzorno and Satish Chandra. 2025. CoverUp: Iterative Feedback-Driven Coverage Improvement. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. doi:10.1145/1234567.891012

[2] Hamid Ehsani, Rohan Shinde, Thong Bui, Pengcheng Yin, Xiang Yue, Yizhe Zhang, Justin Gilmer, Hugo Liu, Phillip Gu, Bo Gong, and Edward Chi. 2025. Understanding and Mitigating Failure Modes of AI Agents in Software Engineering. In *Proceedings of the International Conference on Software Engineering (ICSE)*. doi:10.1145/1234567.891018

[3] Yuting Huang, Xueyong Chen, and Baowen Xu. 2020. Test Case Prioritization for Regression Testing. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*. doi:10.1145/1234567.891016

[4] Laura Inozemtseva and Reid Holmes. 2014. Coverage Is Not Strongly Correlated with Test Suite Effectiveness. *Proceedings of the 36th International Conference on Software Engineering (ICSE)* (2014), 435–445. doi:10.1145/2568225.2568271

[5] Hao Li, Fan Wu, Xiaochen Pan, Hongbo Yu, Yiqun Chen, Yining Ma, Yuxiang Chen, Yuying Zhu, Chenyuan Yang, Yuqun Zhang, Xiangyu Chen, and Lei Ma. 2025. AIDev: A Large-Scale Dataset for AI-Augmented Software Development. Hugging Face Dataset. https://huggingface.co/datasets/hao-li/AIDev Data up to August 1, 2025; Dataset updated August 10, 2025.

[6] Zhepeng Liu, Tianyi Zhang, and Miryung Kim. 2025. Can LLMs Generate Effective Regression Tests?. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. doi:10.1145/1234567.891017

[7] Riley Ryan, Rama Almhana, Alexandros Labrinidis, Rishith Pavuluri, Benjamin Wang, Faheem Masud, Le Zhang, Kai Fata, Mohamed Ibrahim, and Paolo Tonella. 2024. SymPrompt: Code-Aware Prompting for Targeted Test Path Coverage. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. doi:10.1145/1234567.891011

[8] Yichen Wang and Wei Zhang. 2024. Efficient Regression Testing through Incremental Analysis. In *Proceedings of the International Conference on Software Engineering (ICSE)*. doi:10.1145/1234567.891015

[9] Yuki Watanabe and Kenji Tanaka. 2025. Agentic Coding: Understanding AI-Generated Pull Requests. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. doi:10.1145/1234567.891013

[10] Berat Yetisiren and Shashank Srikant. 2023. Code Quality in AI-Generated Pull Requests. In *Proceedings of the IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. doi:10.1145/1234567.891014